Root registry

Status: implemented. The execution-root registry is `RootKind`, `rootKinds`, `getRoot`, and `setRoot` in `components/core/runtime-types.fs`.

What the registry is for

A root is an execution-state pointer into an append-only association vector in the store: the vocabulary, user words, bindings, sets, declarations, generic functions, word table, history, classes, instance states, and channels. Before the registry, every caller that persisted or re-threaded a root named its field directly. A new root could be added and silently omitted from one of those call sites.

`RootKind` is the one closed list. `rootKinds` is the ordered enumeration; `getRoot` reads one; `setRoot` writes one and returns a named refusal if a caller tries to remove the mandatory vocabulary root. The registry is the only place that enumerates the association roots.

Contract

- A root is an `EntryIdentity option` except the vocabulary root, which is always present. `getRoot RootKind.VocabularyRoot` always returns `Some`.
- `setRoot RootKind.VocabularyRoot None` is the named refusal `the vocabulary root cannot be absent`.
- `rootKinds` contains every `RootKind` case exactly once.
- Roots are execution state. Persistence serializes the store only; it does not serialize the registry, a frame, the scheduler, or ownership state. A reader reattaches roots explicitly with `setRoot`, then reads the store.
- Adding a root means adding a `RootKind` case, adding it to `rootKinds`, and extending `getRoot` and `setRoot`. The coverage test fails until all four are done.
- The current and saved frames are control state, not registry roots; they are not persisted or re-threaded from the image.
- `SchedulerState.ChannelsRoot` is part of the registry because it is an association root in execution state, even though it lives on the scheduler record.

What is not claimed

The registry does not make execution state durable, does not serialize a root, does not replace the store's image format, and does not give a language word access to a root. It is a maintainer-facing enumeration and a persistence boundary.

Evidence

The suite checks three things:

- every `RootKind` case appears in `rootKinds`, discovered by reflection;
- every kind round-trips through `setRoot` and `getRoot`;
- every kind's target entry survives `Persistence.toBytes`/`fromBytes`, and a fresh state can reattach it.